

👉 XAI Lecture 09

Adversarial Attacks on Explanations

FAMAF - UNC

First Semester 2026



eXplainable
Artificial Intelligence

Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

<https://interpretable-ml-class.github.io/>

Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods

Dylan Slack*
University of California, Irvine
dslack@uci.edu

Sophie Hilgard*
Harvard University
ash798@g.harvard.edu

Emily Jia
Harvard University
ejia@college.harvard.edu

Sameer Singh
University of California, Irvine
sameer@uci.edu

Himabindu Lakkaraju
Harvard University
hlakkaraju@seas.harvard.edu

ABSTRACT

As machine learning black boxes are increasingly being deployed in domains such as healthcare and criminal justice, there is growing emphasis on building tools and techniques for explaining these black boxes in an interpretable manner. Such explanations are being leveraged by domain experts to diagnose systematic errors and underlying biases of black boxes. In this paper, we demonstrate that post hoc explanations techniques that rely on input perturbations, such as LIME and SHAP, are not reliable. Specifically, we propose a novel *scaffolding* technique that effectively hides the biases of

Explanation Methods. In *Proceedings of the 2020 AAAI/ACM Conference on AI, Ethics, and Society (AIES '20)*, February 7–8, 2020, New York, NY, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3375627.3375830>

INTRODUCTION

Owing to the success of machine learning (ML) models, there has been an increasing interest in leveraging these models to aid decision makers (e.g., doctors, judges) in critical domains such as healthcare and criminal justice. The successful adoption of these models in domain-specific applications relies heavily on how well

GJ 3 Feb 2020

(Slack et al. 2020)

Motivation ---

- ML is increasingly used for **critical** decisions
 - ▶ Healthcare, criminal justice, finance
- Decision makers must **understand model behavior** to:
 - ▶ Diagnose errors and potential biases
 - ▶ Decide when and how much to trust these models
- This motivates the need for **explainable AI (XAI)** tools
 - ▶ e.g., LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017)



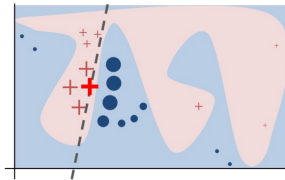
Motivation ---

- Trade-off between interpretability and accuracy
 - ▶ **Simple** models can be easily **interpreted** (e.g., linear regression)
 - ▶ **Complex** but also black-box model has much **better performance** (e.g., deep neural network)
- Can a ML method be both **interpretable** and **accurate**?
- *Post hoc* explanation can seemingly solve this problem:
 - ▶ First build **complex and accurate** ML models for good performance
 - ▶ Then use post hoc explanation for model **interpretation**
- The question is:
 - ▶ How robust and reliable is the *post hoc* explanation methods?

Contribution: A Framework to 'Fool' Post Hoc Explanations ---

- A **scaffolding** framework that hides the biases of any black box classifier
 - ▶ Targets **perturbation-based** explanation methods (LIME & SHAP)
 - ▶ Lets an adversary craft arbitrary, innocuous-looking explanations
- Evaluated on real-world datasets with extremely biased classifiers
 - ▶ COMPAS, Communities & Crime, German Credit
- **Key takeaway:** LIME and SHAP are **not robust enough** to detect discriminatory behavior in adversarial settings

🍷 Perturbation-based Post Hoc Explanation Method ---



Preliminaries & Background

$$\arg \min_{g \in \mathcal{G}} L(f, g, \pi_x) + \Omega(g)$$

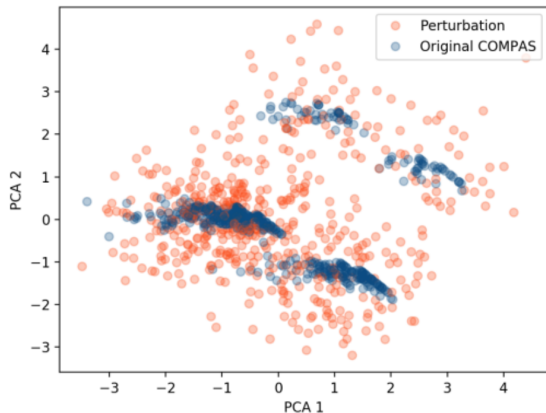
where the loss function L is defined as:

$$L(f, g, \pi_x) = \sum_{x' \in X'} [f(x') - g(x')]^2 \pi_x(x')$$

- f is the original classifier and x is the datapoint we want to explain
- g is the explanation we want to learn, $\Omega(g)$ is the “complexity” of g
- π is the proximity measure
- X' is a **synthetic dataset**, consisting of perturbations of x



Intuition ---



- LIME and SHAP explain predictions by **perturbing** input instances
- These perturbations often fall **outside the real data distribution** (out-of-distribution, OOD)
- Key observation: OOD points are **easily distinguishable** from real data

If we can detect whether a point is a perturbation or real data, we can make the model behave differently on each — and fool the explainer.

🍷 Approach: Set-up ---

Adversary would like to deploy a biased classifier f !

- **Background:** the biased model f uses sensitive attributes to make critical decisions
- **Requirement:** give access of black-box models to customers and regulators who use post-hoc explanations
- **Goal:** hide bias of the classifier f



Approach: Set-up ---

Goal: build an adversarial classifier $\tilde{e}$ that wraps any biased model f

- **Input:** dataset X sampled from the real-world distribution $\mathcal{X}_{dist}$
- **Output:** adversarial classifier $\tilde{e}$ defined as:

$$\tilde{e}(x) = \begin{cases} f(x) & \text{if } x \in \mathcal{X}_{dist} \quad (\text{real data} \rightarrow \text{biased behavior}) \\ \psi(x) & \text{otherwise} \quad (\text{perturbations} \rightarrow \text{innocuous behavior}) \end{cases}$$

where f is the **biased** classifier and ψ is an **unbiased** classifier

It's a Trap

On real data, $\tilde{e}$ behaves like f . On OOD perturbations, it looks fair.

Approach: OOD Detection ---

Key question: given input x , does it come from $\mathcal{X}_{dist}$ or is it a perturbation?

Solution: train a binary classifier `is_OOD(x)` using synthetic labels:

Data	Label
Original instances X	<code>False</code> (real data)
Perturbed instances X_p	<code>True</code> (OOD) — unless too close to X

- Perturbations are generated the same way LIME/SHAP would generate them
- Any off-the-shelf classifier works (authors use Random Forest)

Simple explained

`is_OOD(x)` is the gatekeeper: it decides whether $\tilde{e}$ runs f or ψ



How hard can it be to implement this? 🍏 ---

Step 1: Generate perturbations (same method as LIME/SHAP)

```
X_perturbed = perturb(X)
```

Step 2: Remove perturbations too close to real data

```
X_ood = [x for x in X_perturbed if not too_close(x, X)]
```

Step 3: Build labeled dataset

```
X_train = X + X_ood
```

```
y_train = [False] + [True]    # False = real, True = OOD
```

Step 4: Train OOD detector

```
is_OOD = RandomForest().fit(X_train, y_train)
```

Step 5: Build adversarial classifier

```
def e(x):
```

```
    if not is_OOD(x):    # real data → biased behavior
```

```
        return f(x)
```

```
    else:                # perturbation → innocuous behavior
```

```
        return psi(x)
```

🍌 How hard can it be to implement this? 🤖 ---

Step 1: Generate perturbations (same method as LIME/SHAP)

```
X_perturbed = perturb(X)
```

Step 2: Remove perturbations too close to real data

```
X_ood = [x for x in X_perturbed if not too_close(x, X)]
```

Step 3: Build labeled dataset

```
X_train = X + X_ood
```

```
y_train = [False] + [True]    # False = real, True = OOD
```

Step 4: Train OOD detector

```
is_ood = RandomForest().fit(X_train, y_train)
```

Step 5: Build adversarial classifier

```
def e(x):
```

```
    if not is_ood(x):    # real data → biased behavior
```

```
        return f(x)
```

```
    else:                # perturbation → innocuous behavior
```

```
        return psi(x)
```

Now for real! 🤖

Let's ask an AI agent to code it properly — using the scikit-learn predictor interface

[🔗](#) adversarial_scaffold.py 🤖

Experiment: Set-up ---

90% training & 10% test

Dataset	Size	Features	Positive Class	Sensitive Feature
COMPAS	6172	<i>criminal history, demographics, COMPAS risk score, jail and prison time</i>	High Risk (81.4%)	African-American (51.4%)
Communities & Crime	1994	<i>race, age, education, police demographics, marriage status, citizenship</i>	Violent Crime Rate (50%)	White Population (continuous)
German Credit	1000	<i>account information, credit history, loan purpose, employment, demographics</i>	Good Customer (70%)	Male (69%)

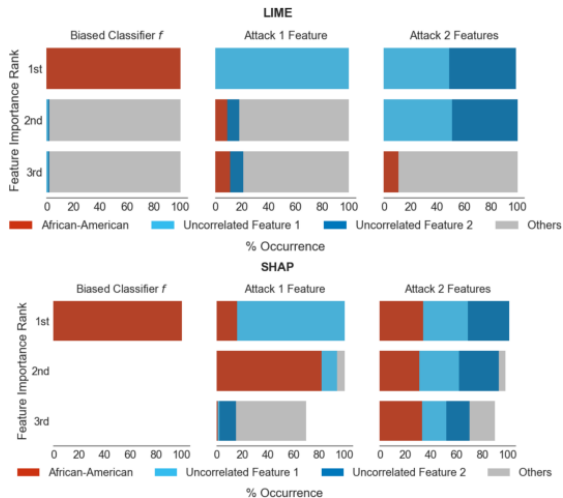
- Biased classifier $\mathbf{f}$ makes predictions purely based on sensitive attributes (race, gender)
- Unbiased classifier ψ uses only features uncorrelated with the sensitive attributes

Social Biases

We are not looking for a flaw in the mechanics of the calculation, but for the persistence of historical patterns that arbitrarily prioritize or exclude certain groups

🍷 Experiment: Results -- COMPAS ---

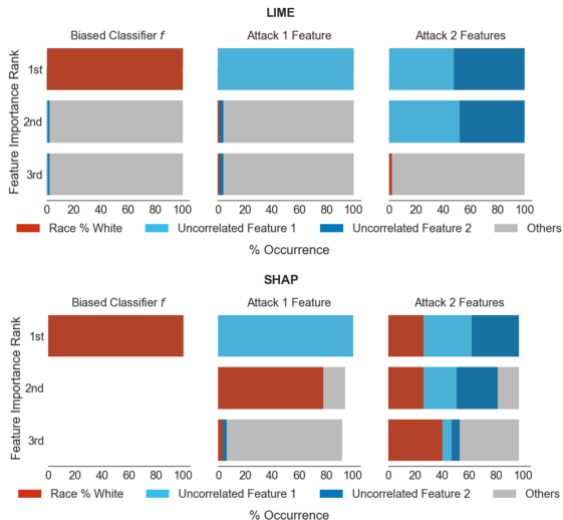
- Each bar shows how often a feature appears at a given importance rank (1st, 2nd, 3rd) across test instances
- **LIME**: the attack is fully successful — race disappears from all explanations in 100% of instances
- **SHAP**: effective with 1 feature (84%), but less so with 2 — SHAP's local accuracy property distributes attribution across features, making it harder to concentrate on a single decoy



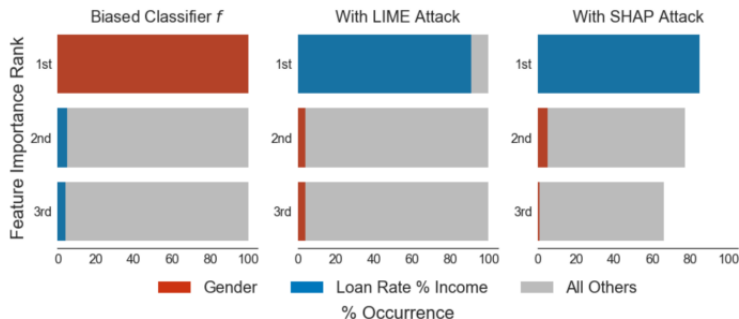


Experiment: Results -- Communities and Crime ---

- Each bar shows how often a feature appears at a given importance rank (1st, 2nd, 3rd) across test instances
- LIME:** the attack succeeds in both settings — race is effectively hidden in nearly 100% of instances
- SHAP:** partially effective with 1 feature, but race re-emerges at lower ranks with 2 features — consistent with SHAP's local accuracy constraint distributing attribution more broadly



🍷 Experiment: Results -- German Credit ---



- Here ψ uses a **real feature** (Loan Rate % Income) instead of a synthetic uncorrelated one
- Both LIME and SHAP attacks succeed: gender disappears from the top-3 in ~91% and ~85% of instances respectively
- Using a real feature as a decoy produces the cleanest attack — it is more natural and harder to detect

Takeaway from Experiments ---

- ① Accuracy of the OOD classifier $\rightarrow$ success of the adversarial attack
- ② Adversarial classifiers to LIME are ineffective against SHAP explanations
 - Ⓐ Any sufficiently accurate OOD classifier is sufficient to fool LIME, while fooling SHAP requires more accurate classifiers
- ③ SHAP less successful when using two features $\leftarrow$ local accuracy property
 - Ⓐ Distribute attributions among several features

Related Works ---

- Issues with post-hoc explanations:
 - ▶ **Doshi-Velez and Kim** identify explainability of predictions as a potentially useful feature of interpretable models.
 - ▶ **Lipton** and **Rudin** argues post-hoc explanations can be misleading and are not trustworthy for sensitive applications.
 - ▶ **Ghorbani et al.** and **Mittelstadt et al.** identified further weaknesses of post-hoc explanations.
- Adversarial explanations
 - ▶ **Dombrowski et al.** and **Heo et al.** show how to change saliency maps in arbitrary ways by imperceptibly changing inputs.

- Are the experimental results sufficient to justify the conclusions?
 - ▶ In particular, how can we explain the discrepancy in results for LIME vs. SHAP?
- What about fooling other classes of post-hoc explanation methods?
 - ▶ Past work: gradient-based methods
- Alternatively, can one design post-hoc explanations that are *adversarially robust*?

Explanations can be manipulated and geometry is to blame

Ann-Kathrin Dombrowski¹, Maximilian Alber¹, Christopher J. Anders¹,
Marcel Ackermann², Klaus-Robert Müller^{1,3,4}, Pan Kessel¹

¹Machine Learning Group, EE & Computer Science Faculty, TU-Berlin

²Department of Video Coding & Analytics, Fraunhofer Heinrich-Hertz-Institute

³Max Planck Institute for Informatics

⁴Department of Brain and Cognitive Engineering, Korea University
{klaus-robert.mueller, pan.kessel}@tu-berlin.de

Abstract

Explanation methods aim to make neural networks more trustworthy and interpretable. In this paper, we demonstrate a property of explanation methods which is disconcerting for both of these purposes. Namely, we show that explanations can be manipulated *arbitrarily* by applying visually hardly perceptible perturbations to the input that keep the network's output approximately constant. We establish theoretic

(Dombrowski et al. 2019)

Motivation ---

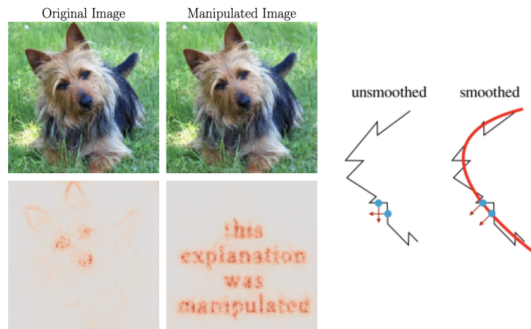
- Explanation methods aim to make ML models more **trustworthy** and **interpretable**
 - ▶ Understand and verify model behavior
 - ▶ Aid decision making in high-stakes scenarios (healthcare, criminal justice)
- This requires **reliable** explanations

Can we always trust model explanations?

This paper shows **the answer is no** — explanations can be manipulated arbitrarily with imperceptible input perturbations, while keeping the model's output unchanged.

🍷 Summary / Contribution 1/2 ---

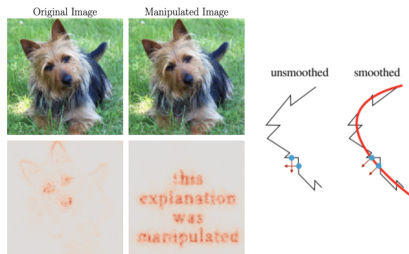
- Manipulate explanations! 🐱
- Provide a theoretical understanding of such nonrobustness and derive a bound 🤔
- Introduce smoothing to increase explanation robustness! 😎



The key insight of the entire paper

The gradient is highly sensitive to small input changes — like a model that memorizes instead of generalizing, it **overfits** to each point on the manifold and changes drastically between neighbors.

🍵 Summary / Contribution 2/2 ---



The bound makes this precise: the change in explanation is proportional to the **curvature** of the manifold:

$$\|h(p) - h(p_0)\| \leq |\lambda_{\max}| d_g(p, p_0)$$

- $|\lambda_{\max}|$ — curvature: how severely the gradient overfits locally
- $d_g(p, p_0)$ — geodesic distance: how far apart the two inputs truly are on the manifold

Background + Related Work ---

- **Interpretation of Neural Networks is Fragile** — Ghorbani et al. (2019)
 - ▶ Small input perturbations cause unstructured, unpredictable changes in explanation maps
- **The (Un)reliability of Saliency Methods** — Kindermans et al.
 - ▶ Explanations should be invariant to constant input shifts — many methods fail this basic test
- **Sanity Checks for Saliency Maps** — Adebayo et al.
 - ▶ Randomizing network weights should destroy explanations — surprisingly, some methods are insensitive
- **Fairwashing with Off-Manifold Detergent** — Anders et al.
 - ▶ Biased models can hide their behavior by exploiting the gap between the true data manifold and the high-dimensional embedding space

Model

A neural network $g : \mathbb{R}^d \rightarrow \mathbb{R}^K$ with ReLU non-linearities

- Predicted class: $k = \arg \max_i g(x)_i$
- Explanation map: $h : \mathbb{R}^d \rightarrow \mathbb{R}^d$ — assigns a relevance score to each pixel

Goal of the attack:

Given input x , find a manipulated image $x_{adv} = x + \delta x$ such that:

- ① $h(x_{adv}) \approx h^t$ — explanation matches an arbitrary target map
- ② $g(x_{adv}) \approx g(x)$ — network output stays unchanged
- ③ $\|\delta x\| \ll 1$ — perturbation is imperceptible

🍷 Properties of Manipulated Image ---

The attack constructs $x_{adv} = x + \delta x$ satisfying three conditions simultaneously:

- 1 **Prediction unchanged:** $g(x_{adv}) \approx g(x)$ — the model still outputs the same class with the same confidence
- 2 **Explanation hijacked:** $h(x_{adv}) \approx h^t$ — the explanation now shows whatever the attacker wants, regardless of the true model behavior
- 3 **Perturbation imperceptible:** $\|\delta x\| \ll 1$ — the modified image looks identical to the original

The Explanation Methods Under Attack ---

Gradient-based

- **Vanilla Gradients:** $h(x) = \frac{\partial g}{\partial x}(x)$
 - ▶ Quantifies how infinitesimal perturbations in each pixel change the prediction
- **Gradient $\times$ Input:** $h(x) = x \odot \frac{\partial g}{\partial x}(x)$
 - ▶ For linear models, gives the exact contribution of each pixel to the prediction
- **Integrated Gradients:** $h(x) = (x - \bar{x}) \odot \int_0^1 \frac{\partial g(\bar{x} + t(x - \bar{x}))}{\partial x} dt$
 - ▶ Accumulates gradients along the path from a baseline $\bar{x}$ to the input x

Propagation-based

- **Guided Backpropagation (GBP):** gradient explanation with negative components zeroed out during backprop
- **Layer-wise Relevance Propagation (LRP):** propagates relevance scores backwards through the network layers
- **Pattern Attribution (PA):** standard backpropagation with weights scaled by learned patterns

Manipulation Method 1/3 ---

Obtain manipulated images by optimizing the loss function:

$$\mathcal{L} = \|h(x_{adv}) - h^t\|^2 + \gamma \|g(x_{adv}) - g(x)\|^2$$

with respect to x_{adv} using gradient descent

- $h(x_{adv})$: manipulated explanation map
- h^t : target map
- γ : weighting hyperparameter (free parameter)
- $g(x_{adv})$: network output (manipulated input)
- $g(x)$: network output (original input)

Manipulation Method 2/3 ---

Obtain manipulated images by optimizing the loss function:

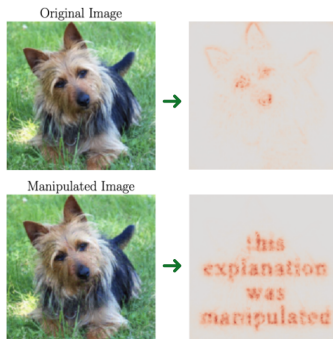
$$\mathcal{L} = \|h(x_{adv}) - h^t\|^2 + \gamma \|g(x_{adv}) - g(x)\|^2$$

- **First term:** penalizes deviation from the target map h^t — the optimizer wants the explanation to match exactly what the attacker chose
- **Second term:** penalizes any change in the network output — the optimizer wants the prediction to stay identical
- γ : controls the trade-off
 - ▶ Large $\gamma \rightarrow$ preserving the prediction takes priority
 - ▶ Small $\gamma \rightarrow$ manipulating the explanation takes priority, even at the cost of slightly changing the output

🍷 Manipulation Method 3/3 ---

Obtain manipulated images by optimizing the loss function:

$$\mathcal{L} = \|h(x_{adv}) - h^t\|^2 + \gamma \|g(x_{adv}) - g(x)\|^2$$



Find an image that looks identical to the original, predicts the same class, but whose explanation says whatever the attacker wants.

🍷 Manipulation Method: Softplus Trick ---

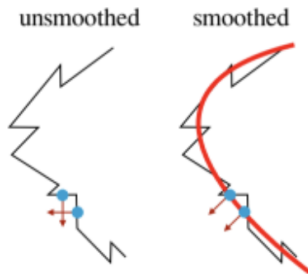
To minimize $\mathcal{L}$ with gradient descent, we need to compute the gradient of the loss with respect to x_{adv} . But that gradient involves the second derivative of the network:

$$\partial_{x_{adv}} \|h(x_{adv}) - h^t\|^2 \propto \frac{\partial h}{\partial x_{adv}} = \frac{\partial^2 g}{\partial x_{adv}^2} \propto \text{relu}'' = 0$$

ReLU has second derivative exactly zero everywhere — there is no signal for the optimizer to follow.

Solution: replace ReLU with softplus during optimization:

$$\text{softplus}_{\beta}(x) = \frac{1}{\beta} \log(1 + e^{\beta x})$$



Softplus is a smoothed version of ReLU, nearly identical for large β , but with a well-defined second derivative. Once optimization is complete, x_{adv} is tested on the original ReLU network.

Experiments: Setup ---

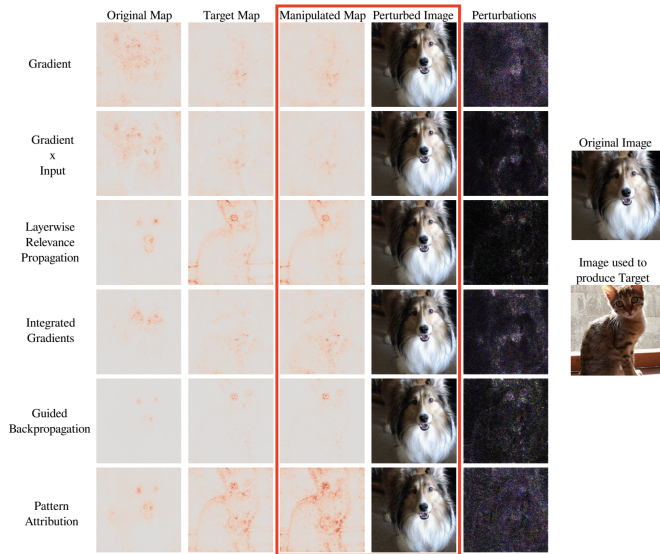
- 100 randomly selected images per explanation method
- Model: **VGG-16** pre-trained on ImageNet

For each run:

- ① Pick two random test images
- ② Use the first to generate a **target explanation map** h^t
- ③ Perturb the second with gradient descent until its explanation matches h^t

Results are consistent across **ResNet-18**, **AlexNet**, **DenseNet-121** and the **CIFAR-10** dataset

🍷 Results: Visual Comparison ---

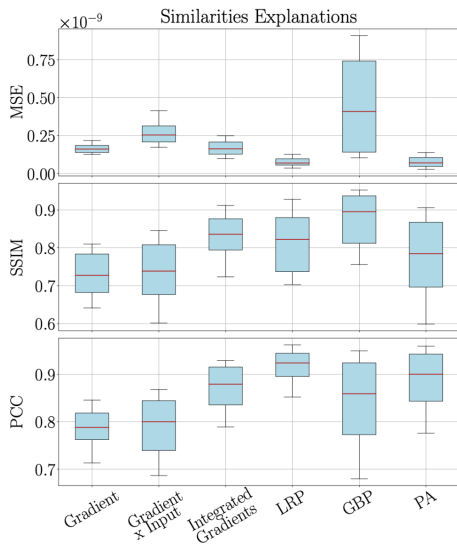


The attack is applied to a dog image using the cat's explanation as target h^t , across all six explanation methods.

- **Manipulated Map** closely matches the **Target Map** in every row — the attack succeeds for all methods
- **Perturbed Image** is visually identical to the original — the perturbation δx is imperceptible
- The red box highlights what the user actually sees: a normal image with a completely fabricated explanation

A user inspecting any of these explanations would have no way of detecting the manipulation.

🍷 Results: Quantitative Metrics 1/2 ---

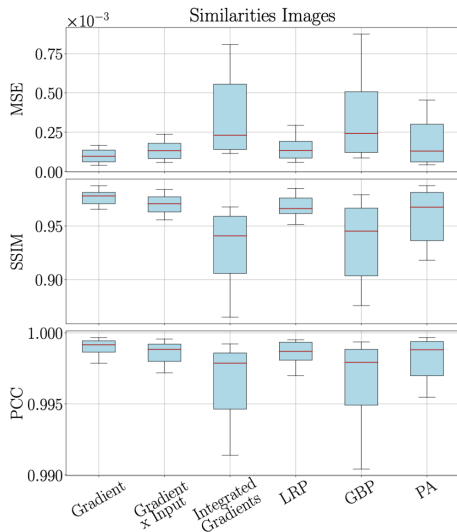


Similarity between manipulated explanation and target h^t :

- MSE near zero ($\times 10^{-9}$), SSIM in $[0.75, 0.9]$, PCC in $[0.8, 0.95]$ across all methods
- The manipulated explanation closely matches the target in every case
The attack consistently achieves its goal across all 6 explanation methods.

Note: GBP shows higher variance than the rest, suggesting it is slightly harder to attack consistently, but the attack still succeeds.

🍷 Results: Quantitative Metrics 2/2 ---



Similarity between perturbed image and original:

- MSE near zero ($\times 10^{-3}$), SSIM in $[0.9, 0.98]$, PCC in $[0.995, 0.999]$ across all methods
- The perturbed image is virtually indistinguishable from the original
The attack is invisible to the human eye — while the explanation says whatever the attacker wants.

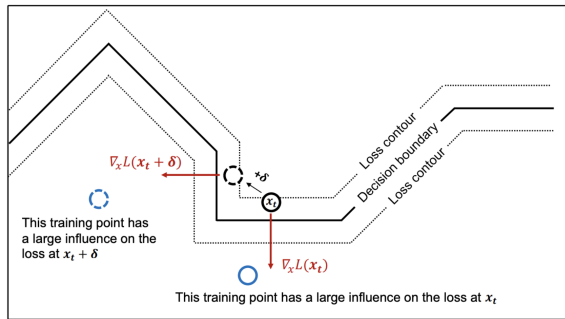
🍷 Theoretical Analysis Intuition ---

Why Are Explanations Vulnerable?

Because of the large curvature of the neural network's output manifold.

The decision boundary of a ReLU network is piecewise linear — full of sharp kinks. A tiny perturbation $+\delta$ moves x_t to $x_t + \delta$ along the boundary, but the gradient $\nabla_x L$ can change direction drastically:

- **At** x_t : the gradient points downward
- **At** $x_t + \delta$: the gradient points left



(Ghorbani, Abid, and Zou 2019)

Two nearly identical inputs — completely different explanations.

This is the geometric root of the vulnerability.

Theoretical Bound ---

Theorem 1

Let $g : \mathbb{R}^d \rightarrow \mathbb{R}$ be a network with softplus_β non-linearities and $\mathcal{U}_\epsilon(p) = \{x \in \mathbb{R}^d; \|x - p\| < \epsilon\}$ an environment of a point $p \in S$ such that $\mathcal{U}_\epsilon(p) \cap S$ is fully connected. Let g have bounded derivatives $\|\nabla g(x)\| \geq c$ for all $x \in \mathcal{U}_\epsilon(p) \cap S$. It then follows for all $p_0 \in \mathcal{U}_\epsilon(p) \cap S$ that

$$\|h(p) - h(p_0)\| \leq |\lambda_{\max}| d_g(p, p_0) \leq \beta C d_g(p, p_0)$$

where $\lambda_{\max}$ is the principle curvature with the largest absolute value for any point in $\mathcal{U}_\epsilon(p) \cap S$ and the constant $C > 0$ depends on the weights of the neural network.

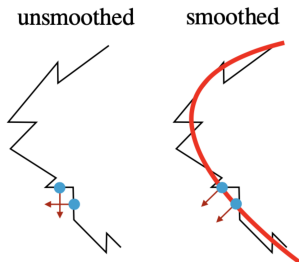
In Plain English

- Two inputs that produce the **same prediction** can have very different explanations
- The difference is not arbitrary — it is bounded by the **curvature** of the output manifold
- High curvature means the explanation can change drastically with a tiny perturbation
- The curvature is controlled by β : **smaller $\beta \rightarrow$ flatter manifold $\rightarrow$ more stable explanations**
- The theorem does not just explain *why* explanations are fragile — it tells us exactly *what to fix*

🍷 Robustness via Smoothing ---

Idea: replace ReLU with softplus_β to smooth the sharp kinks of the output manifold

$$\text{softplus}_\beta(x) = \frac{1}{\beta} \log(1 + e^{\beta x})$$



- **Unsmoothed (ReLU):** the manifold is piecewise linear — full of kinks where the gradient changes direction abruptly, even between neighboring points
- **Smoothed (softplus):** the manifold becomes a smooth curve — nearby points now have similar gradients, and therefore similar explanations
- The model itself is **not modified** — smoothing is only applied during the explanation process

Smaller $\beta \rightarrow$ smoother manifold $\rightarrow$ harder to manipulate the explanation

Smoothing: Connections to SmoothGrad ---

Theorem 2

For a one-layer neural network $g(x) = \text{relu}(w^T x)$ and its β -smoothed counterpart $g_\beta(x) = \text{softplus}_\beta(w^T x)$, it holds that

$$\mathbb{E}_{\epsilon \sim p_\beta} [\nabla g(x - \epsilon)] = \nabla g_{\frac{\beta}{\|w\|}}(x)$$

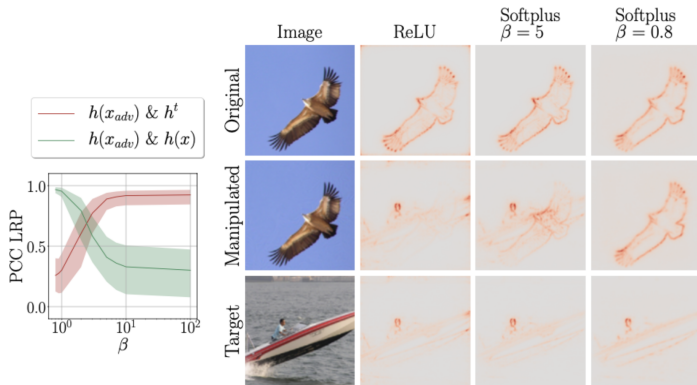
where $p_\beta(\epsilon) = \frac{\beta}{(e^{\beta\epsilon/2} + e^{-\beta\epsilon/2})^2}$.

Both methods make explanations more robust by **averaging out** the sharp kinks of the manifold, they just do it differently:

- **SmoothGrad**: adds random noise to the input many times and averages the resulting gradients
- **β -smoothing**: replaces ReLU with softplus — mathematically equivalent, but in a single pass

Same robustness, 50x cheaper to compute

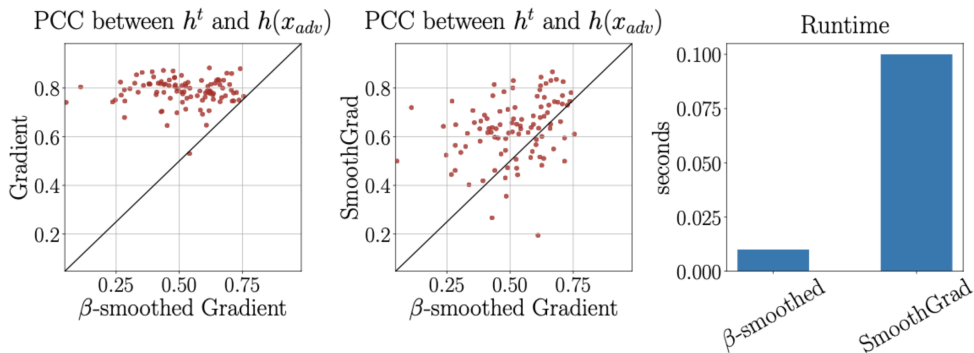
🍷 Robustness Experiments ---



- **Red line:** correlation between the manipulated explanation and the target h^t — drops as β decreases
- **Green line:** correlation between the manipulated explanation and the original $h(x)$ — rises as β decreases
- **The crossing point** marks where the attack stops working — around $\beta \approx 3$
- **Image grids:** with ReLU the attack succeeds perfectly; with softplus $\beta = 5$ it weakens; with softplus $\beta = 0.8$ the manipulated map is virtually identical to the original — the attack is defeated

Reducing β recovers the true explanation and destroys the manipulation, without modifying the model itself

🍷 Robustness Experiments ---



β -smoothing

- 1 Makes explanations more robust.
- 2 Is comparable to SmoothGrad.
- 3 Has a faster runtime than SmoothGrad.

Conclusion - Critique ---

Strengths

- Thorough investigation: problem $\rightarrow$ reason $\rightarrow$ solution
- Extensive validation: various explanation methods, models, and datasets

Limitations

- Analyses focused on relu/softplus activation function
- Evaluation of robustness based on Pearson correlation coefficient

🍷 Future Directions & Food for Thought ---

Future directions

- Extend empirical analyses to other tasks and data modalities
- Generalize theoretical analyses to propagation-based methods
- Modify model training process to make NNs less vulnerable to explanation manipulation
 - ▶ Low-curvature models [Srinivas et al., NeurIPS 2022]

Food for thought

- Might there be other reasons for explanations being sensitive to manipulation?
- What are other ways to evaluate robustness of explanations?
- Is it a good idea to trade-off faithfulness for better robustness?

Thank You!



References --- |

- Dombrowski, Ann-Kathrin, Maximilian Alber, Christopher J. Anders, Marcel Ackermann, Klaus-Robert Müller, and Pan Kessel. 2019. “Explanations Can Be Manipulated and Geometry Is to Blame.” In *Advances in Neural Information Processing Systems*.
- Ghorbani, Amirata, Abubakar Abid, and James Zou. 2019. “Interpretation of Neural Networks Is Fragile.” In *Proceedings of the Aaai Conference on Artificial Intelligence*.
- Slack, Dylan, Sophie Hilgard, Emily Jia, Sameer Singh, and Himabindu Lakkaraju. 2020. “Fooling LIME and SHAP: Adversarial Attacks on Post Hoc Explanation Methods.” In *Proceedings of the Aaai/Acm Conference on Ai, Ethics, and Society*.